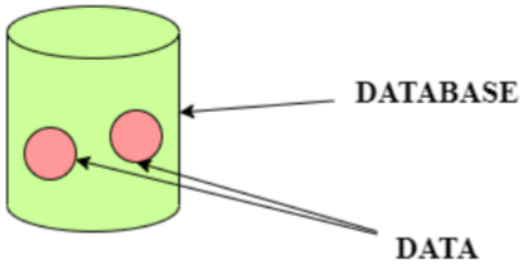
Week 1: Introduction to Databases & Relational Model

1. What is a Database & DBMS?

Database

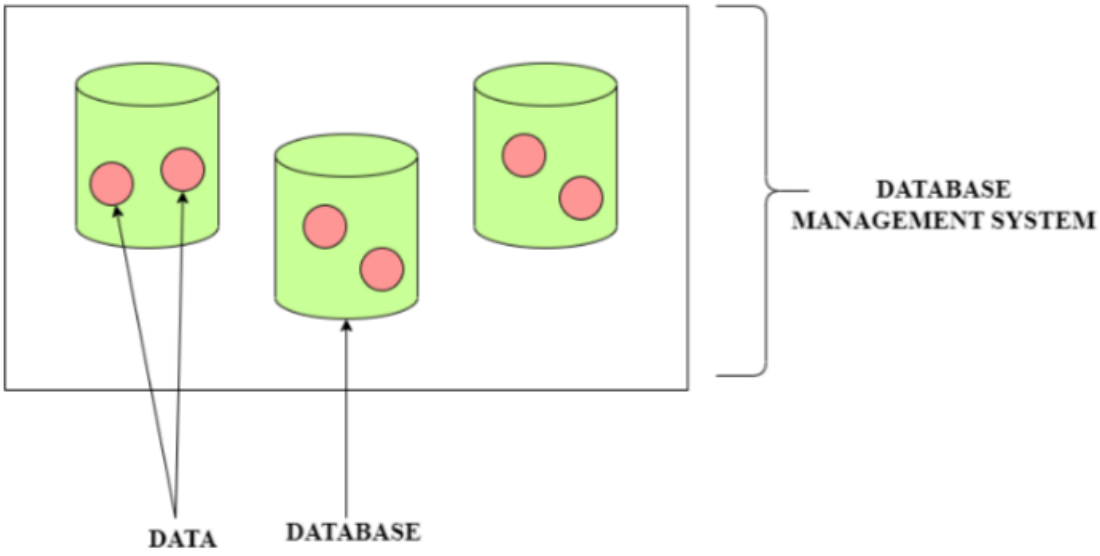
Think of a Database as a digital "filing cabinet." It is an organized collection of structured information.



DBMS (Database Management System)

A DBMS (Database Management System) is the software that acts as the "librarian." You don't grab the files yourself; you ask the librarian (DBMS) to find, add, or delete them for you.

- Examples: MySQL, PostgreSQL, Oracle, Microsoft SQL Server.



Databases are designed to handle large amounts of data efficiently.

Examples:

- Student records in a university system
- Customer data in a banking system
- Products and orders in an e-commerce system

Responsibilities of a DBMS:

- Data storage and retrieval
- Data security and authorization
- Backup and recovery
- Concurrency control
- Data integrity enforcement

2. Role of Databases in Modern Applications

Modern software applications heavily depend on databases to store and manage data.

In the past, apps stored data in simple text files. However, modern apps (like Instagram or Amazon) need databases to:

- **Handle massive amounts of data.**
- **Allow multiple users** to access data at the same time without crashing.

- **Keep data secure** and ensure it doesn't get lost.

Why databases are important:

- Persistent data storage
- Faster data retrieval
- Data consistency and accuracy
- Support for multiple users simultaneously
- Secure access to sensitive data

Real-world usage:

- Web applications (user accounts, orders)
- Mobile applications (profiles, messages)
- Enterprise systems (ERP, CRM)

3. Types of Databases

3.1 Relational Databases (RDBMS)

Relational databases store data in **tables** with rows and columns and use **SQL** for data manipulation.

In RDBMS data is organized into **tables** with fixed rows and columns. They are best for structured data where relationships are important (e.g., Banking systems).

- **Model:** The Relational Model.

Examples: MySQL, PostgreSQL, Oracle

Features:

- Structured schema
- Strong consistency
- Uses relationships between tables

3.2 NoSQL Databases

NoSQL databases are designed for large-scale, distributed, and flexible data storage.

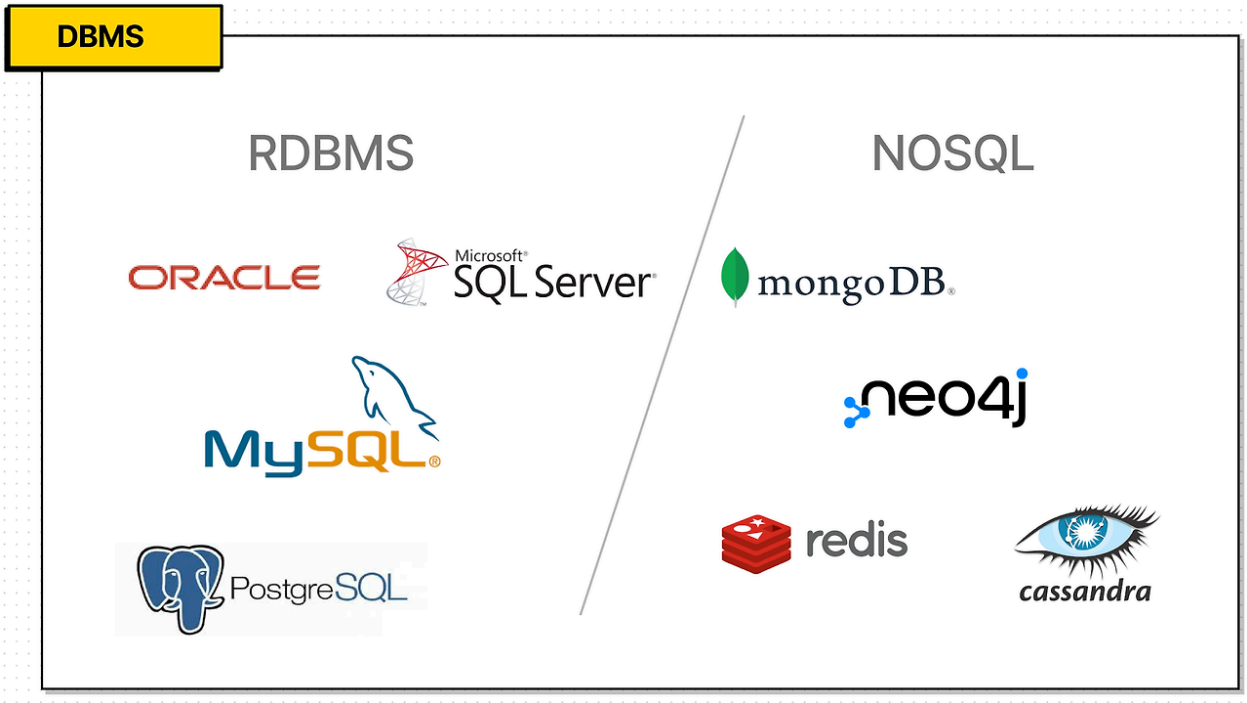
Data is stored in more flexible ways, like folders of documents or simple lists.

- **Document Model:** Data looks like JSON files (e.g., MongoDB).
- **Key-Value Model:** Like a dictionary; a unique "key" points to a "value" (e.g., Redis).

Examples: MongoDB, Redis, Cassandra

Features:

- Schema-less or flexible schema
- High scalability
- Used in big data and real-time systems



4. Database Models

What is a Data Model?

A **data model** defines **how data is structured, stored, related, and accessed** in a database system. It acts as a **blueprint** for designing databases and helps users understand the relationship between different data elements.

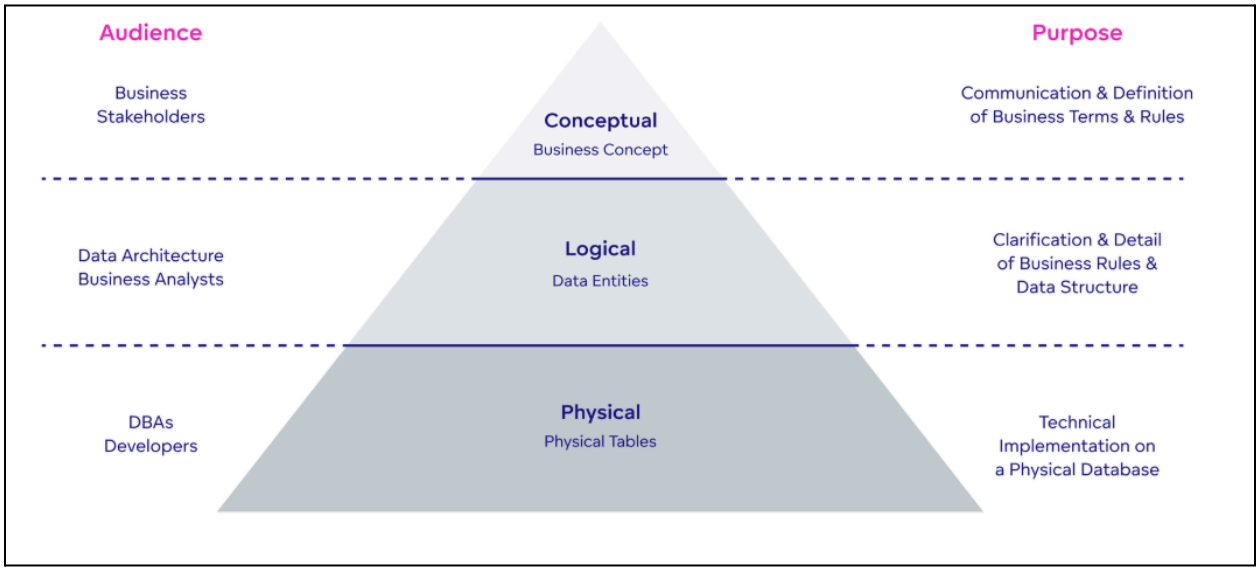
Why Data Models are Important

- Provide a clear structure for data
- Help in database design and development
- Improve data consistency and integrity
- Act as a communication tool between developers and stakeholders

Types of Data Models

Data models can be broadly classified into three categories:

1. Conceptual Data Models
2. Logical Data Models
3. Physical Data Models



4.1 Conceptual Data Model

The conceptual data model represents data at a high level and focuses on ***what data*** *is needed*, not how it is stored.

Characteristics

- Independent of DBMS
- Easy to understand
- Uses business concepts
- Created during requirement analysis

Components

- Entities
- Attributes
- Relationships

Example

- Entity: STUDENT
  Attributes: student_id, name, email

Usage

- Used by business analysts and stakeholders
- Helps understand business requirements

4.2 Logical Data Model

The **logical data model** describes the structure of data in more detail but is still **independent of physical storage**.

Provides more detail about the entire system – its entities, attributes, and relationships. It is used to develop a database schema through the collaboration of data architects and business analysts.

Characteristics

- Defines tables, columns, data types
- Shows relationships and constraints
- Normalized to reduce redundancy

Components

- Tables
- Primary keys
- Foreign keys
- Constraints

Example

STUDENT(student_id PK, name, email)

Usage

- Used by database designers
- Acts as a bridge between conceptual and physical models

4.3 Physical Data Model

The physical data model describes how data is actually stored in the database.

We define exactly how data is stored in the database, including data types, keys, indexes, and constraints.

Characteristics

- DBMS-specific
- Includes indexing and storage details
- Performance-oriented

Components

- Table structures
- Indexes
- Storage engines
- Partitioning

Example

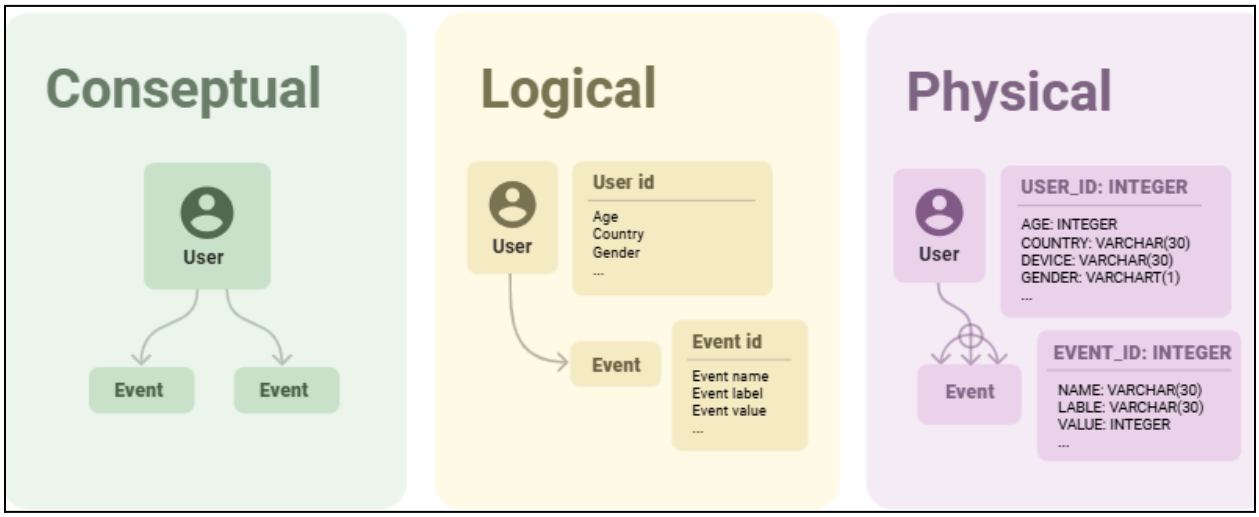
```sql
CREATE TABLE student (
  student_id INT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100) UNIQUE
);
```

Usage

- Used by DBAs and developers
- Optimized for performance

4.4 Summary for all data models

- Data models define how data is organized and stored
- Conceptual, logical, and physical models guide database design
- NoSQL models are used for scalability and flexibility



References for more information…

https://www.owox.com/blog/articles/types-of-data-models-and-benefits

https://www.gooddata.com/blog/what-a-data-model

5. Client–Server Architecture

In database systems, applications follow a **client–server architecture**.

1. **The Server:** A powerful computer (the "Database Server") where the data actually lives.
2. **The Client:** The software you use to talk to the server (like MySQL Workbench or your web browser).

Analogy: You are the *Client* ordering food. The kitchen is the *Server*. The *DBMS* is the waiter taking your order to the kitchen.

5.1 Components of Client–Server Architecture

5.1.1 Client

The client is the front-end application used by users to interact with the database.

Responsibilities of a client:

- Accept user input
- Send SQL queries or requests to the server
- Display query results

Examples of database clients:

- MySQL Workbench
- DataGrip
- pgAdmin
- Web applications (via backend APIs)

5.1.2 Server

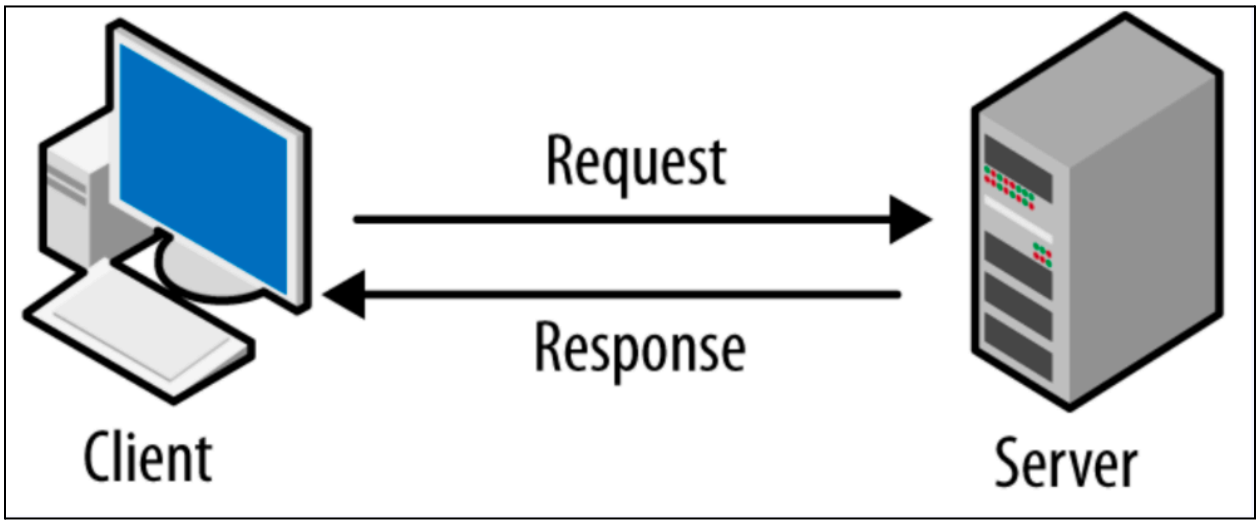
The **server** is the backend system that hosts the database and DBMS software.

Responsibilities of a server:

- Store and manage data
- Execute SQL queries
- Enforce constraints and security rules
- Handle concurrent user access
- Perform backup and recovery

Examples of database servers:

- MySQL Server
- PostgreSQL Server
- Oracle Database Server



5.2. How Client–Server Communication Works

Communication between client and server usually happens through network protocols, mainly TCP/IP.

Steps:

1. Client connects to the database server
2. Client sends SQL query/request
3. Server processes the query
4. Server retrieves or modifies data
5. Server sends results back to the client

5.3 Database as a Server

In DBMS, the database itself runs as a server process that:

- Listens for incoming client connections
- Manages multiple users simultaneously
- Ensures data consistency and integrity

The server runs continuously and serves many clients at the same time.

5.4 Advantages of Client–Server Architecture

- Centralized data management
- Improved data security
- Supports multiple users
- Easier backup and recovery
- Better performance than file-based systems

5.5 Disadvantages of Client–Server Architecture

- Server failure affects all clients
- Network dependency
- Higher setup and maintenance cost
- Requires skilled administration

5.6 Real-World Example

Online Banking System

- Client: Web browser or mobile app
- Application Server: Banking backend system
- Database Server: Stores customer accounts, transactions

This ensures secure, scalable, and reliable data management.

References for more information…

https://youtu.be/ve82kSSj_Hs?si=hxZFcYTLM8x6A-fK

6. Introduction to the Relational Model

6.1 What is the Relational Model?

The **Relational Model** is the most widely used data model in Database Management Systems (DBMS). It organizes data into **tables (relations)** consisting of rows and columns. The model was introduced by **E. F. Codd** and is based on **set theory and predicate logic**.

The relational model ensures:

- Data consistency
- Reduced data redundancy
- Easy data retrieval using SQL

6.2 Components of relational model

6.2.1 Tables (Relations)

A **table** represents a real-world entity and stores data in a structured format.

Characteristics of a Table:

- Has a unique name
- Contains rows and columns
- Each table represents one entity

Example:

STUDENT table

student_id	name	email

6.2.2 Rows (Tuples)

A **row** represents a single record in a table.

Characteristics of Rows:

- Each row contains data for all columns
- Each row must be uniquely identifiable
- No two rows should be identical

Example:

student_id	name	email
101	Kamal	kamal@gmail.com

6.2.3 Columns (Attributes)

A **column** represents a specific attribute or property of an entity.

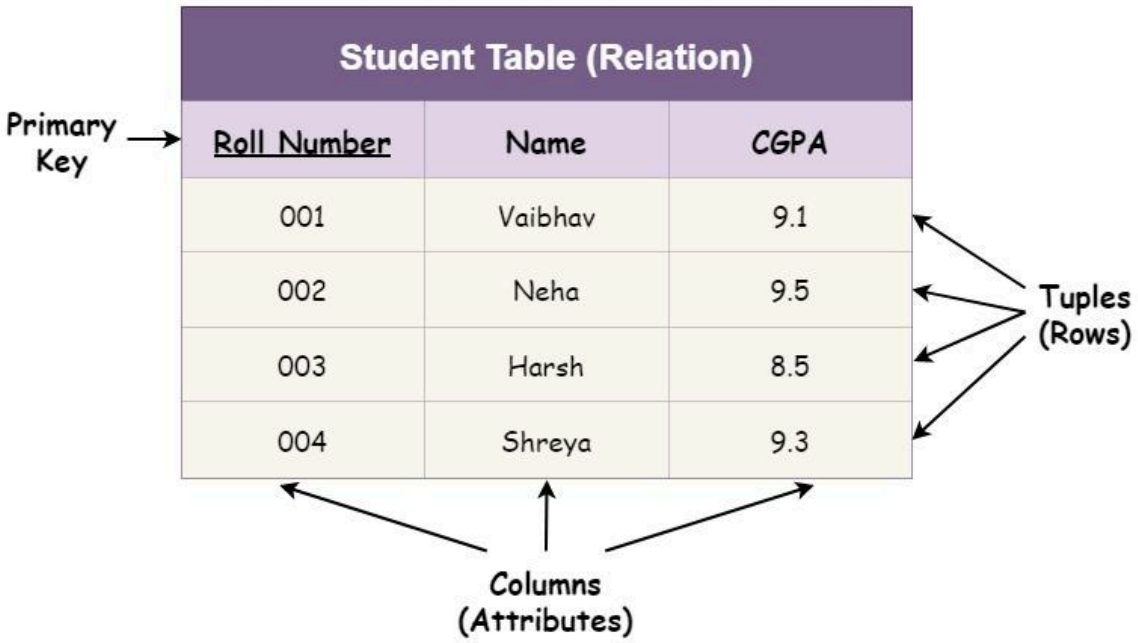
Characteristics of Columns:

- Each column has a name
- Each column has a data type
- Values in a column must be of the same type

Example:

- student_id → Integer
- name → String
- email → String

Summary

- Relational model stores data in tables
- Tables consist of rows and columns

Relational Model in DBMS